

Seminar Paper

An Economic Analysis of MEV Extraction on Uniswap V3: Sandwich Attacks, Bot Profitability, and Victim Losses

Rafael Balasteguim da Silva

Supervised by:

Prof. Dr. Fabian Schär

Credit Suisse Asset Management (Schweiz) Professor for

Distributed Ledger Technologies and Fintech

Center for Innovative Finance, University of Basel

Abstract

This study examines sandwich attacks in a Uniswap V3 pool using on-chain data. The analysis investigates their frequency, profitability, impact on victim traders, and concentration among attackers and block builders. The results show that sandwich attacks are a common phenomenon in decentralized exchanges. Although they create substantial theoretical MEV opportunities, execution frictions such as slippage, pool fees, and gas costs significantly reduce realized profitability. In total, 699 sandwich attacks are detected during the observation period. The results further show that only a minority of attackers are profitable after costs, while victim traders face substantial losses.

Keywords: Decentralized Finance (DeFi), Uniswap V3, Maximal Extractable Value (MEV), Sandwich Attacks

JEL: G10, G23, O33, D47

Contents

1	Introduction	1
2	Theoretical Background	2
2.1	Decentralized Exchanges and Uniswap	2
2.1.1	Automated Market Makers	2
2.1.2	Price Formation Mechanisms	3
2.1.3	Uniswap V3	4
2.2	Maximal Extractable Value	5
2.2.1	Definition of MEV	5
2.2.2	Types of MEV	6
2.2.3	Proposer-Builder Separation (PBS)	6
2.3	Sandwich Attacks	7
3	Data and Methodology	9
3.1	Data	9
3.2	Sandwich Detection	10
3.2.1	Detection Process	10
3.2.2	Sandwich Examples	11
3.2.3	Backrun Classification	13
3.3	Profit Calculation	13
3.4	Victim Loss Measures	15
3.5	Limitations	16
4	Empirical Results	17

4.1	Overview and Descriptive Statistics	17
4.2	Classification of Sandwich Attacks	18
4.3	Bot Profitability	20
4.3.1	Attack Size and Profitability by Backrun Type . .	20
4.3.2	Profit Decomposition	20
4.3.3	Top 10 Attackers	21
4.4	Victim Loss	22
4.4.1	Sandwich Damage	22
4.4.2	Victim Slippage and Execution Loss	23
4.4.3	Top 10 Victims	24
4.5	Attacker-Builder Relationships	25
5	Conclusion	27
5.1	Summary of Findings	27
5.2	Implications	29
5.3	Future Research	30
	References	i
6	Appendix	v

1 Introduction

Decentralized Finance (DeFi) has grown rapidly over the past five years. According to DefiLlama, the total value locked (TVL) in DeFi protocols exceeds USD 70 billion. This shows that DeFi is becoming more important in the financial sector DefiLlama (2026a).

The increasing adoption of decentralized exchanges (DEXs) has also created new opportunities for Maximal Extractable Value (MEV). Because transactions are publicly visible before being included in a block, specialized market participants can exploit transaction ordering to generate profits. One of the most common forms of MEV is the sandwich attack, in which an attacker places transactions immediately before and after a victim’s trade to profit from the resulting price impact Qin et al. (2022).

Previous studies have documented the prevalence of sandwich attacks and their negative effects on traders. Wang et al. further find that many DeFi users underestimate the prevalence and impact of these attacks Wang et al. (2022). However, relatively little is known about the realized profitability of these attacks once execution frictions such as slippage, pool fees, and gas costs are taken into account. Furthermore, the extent to which victim losses correspond to attacker profits remains unclear. Existing studies also provide limited evidence on cases in which sandwich attacks are only partially closed or over closed.

This paper addresses these issues using Ethereum on-chain data from swap events in the Uniswap V3 WETH/USDC 0.05% pool during the first quarter of 2026. The analysis examines the prevalence of sandwich attacks, their profitability, the losses imposed on victim traders, and the degree of concentration among attackers and builders.

This study addresses the following research questions:

- **RQ1:** How prevalent are sandwich attacks in the Uniswap V3 WETH/USDC pool during Q1 2026?
- **RQ2:** How profitable are sandwich attacks, and what factors affect

their profitability?

- **RQ3:** What economic losses do sandwich attacks impose on victim traders?
- **RQ4:** How concentrated is sandwich attack activity, and what relationships exist between MEV bots and block builders?

2 Theoretical Background

2.1 Decentralized Exchanges and Uniswap

DEXs play a central role in the DeFi ecosystem. Traditional exchanges rely on centralized intermediaries and order books to match buyers and sellers. In contrast, DEXs enable users to trade digital assets directly on-chain through smart contracts. This design eliminates the need for a trusted third party. Most modern DEXs use an *Automated Market Maker* (AMM) mechanism, where users trade against liquidity pools instead of traditional order books Xu et al. (2023).

Among DEXs, Uniswap is one of the largest protocols in decentralized finance. According to DefiLlama, Uniswap processed more than USD 41 billion in trading volume during the 30 days prior to 15 June 2026 DefiLlama (2026b).

Given its large market share and high transaction volume, Uniswap V3 is well suited for studying Maximal Extractable Value (MEV) and sandwich attacks, which are the focus of this paper.

2.1.1 Automated Market Makers

Most decentralized exchanges in the DeFi ecosystem are based on Automated Market Maker (AMM) protocols. Instead of trading directly with other market participants, users trade against liquidity pools funded by liquidity providers (LPs). In return for supplying liquidity, LPs earn a

share of the trading fees generated by the pool Xu et al. (2023). Uniswap uses a constant product pricing mechanism, which is discussed in the following section.

2.1.2 Price Formation Mechanisms

The pricing mechanism used in Uniswap is based on the constant product invariant, whereby the product of the reserves of the two assets in the liquidity pool remains constant (Mohan, 2022; Xu et al., 2023):

$$x \cdot y = k \tag{1}$$

Assume that a pool holds $x_0 = 10$ ETH and $y_0 = 500$ USDC. It follows that $k = x_0 \cdot y_0 = 5000$. The initial pool price is therefore $p_0 = y_0/x_0 = 500/10 = 50$ USDC/ETH.

In a first example a trader buys 0.01 ETH from the pool. Since the constant product k must remain unchanged, the reduction in the ETH reserve implies an increase in the USDC reserve. The new reserves therefore become $x_1 = 9.99$ and $y_1 = k/x_1 = 500.5005$. The change in the USDC reserve is $\Delta y = y_1 - y_0 = 500.5005 - 500 = 0.5005$.

Since the trader purchases ETH from the pool, the increase in the USDC reserve corresponds to the amount of USDC that must be paid to the pool. Therefore, the trader pays 0.5005 USDC to obtain 0.01 ETH, corresponding to an execution price of $0.5005/0.01 = 50.05$ USDC/ETH. The new pool price after the transaction is $500.5005/9.99 = 50.10$ USDC/ETH.

Compared to the initial pool price of 50 USDC/ETH, the realized execution price is slightly higher at 50.05 USDC/ETH. This difference between the spot price and the realized execution price is referred to as slippage. In AMMs, every trade experiences some degree of slippage because prices are determined along a continuous pricing curve rather than through matching buy and sell orders Xu et al. (2023). Formally, slippage can be expressed as the difference between the spot price p_0 and the realized

execution price p_e :

$$\text{Slippage} = p_e - p_0 \quad (2)$$

Consider a second example with the same initial reserves of $x_0 = 10$ ETH and $y_0 = 500$ USDC, in which a trader purchases 1 ETH from the pool. The new reserves become $x_1 = 9$ and $y_1 = k/x_1 = 555.5556$. The trader therefore has to pay $555.5556 - 500 = 55.5556$ USDC, corresponding to an execution price of 55.5556 USDC/ETH. The resulting pool price is $555.5556/9 = 61.7284$ USDC/ETH.

Compared to the initial pool price of 50 USDC/ETH, this corresponds to a price increase of approximately 23.46% .

These two examples highlight a central characteristic of constant product AMMs. Small buy orders have only a minor effect on the reserve composition of the pool and therefore cause little price impact. Larger buy orders move the pool price more and therefore lead to higher execution prices. As a result, larger transactions experience greater slippage than smaller transactions Mohan (2022).¹

2.1.3 Uniswap V3

The main innovation introduced by Uniswap V3 is *concentrated liquidity*. In Uniswap V1 and V2, liquidity was distributed across the entire price range from zero to infinity. The problem with this approach was that a large share of the capital deposited by liquidity providers remained unused, since trading activity typically occurs only within a relatively narrow price range. This is especially true for pairs of stablecoins. For example, the exchange rate of USDC/USDT is generally expected to remain close to 1:1, making it inefficient to provide liquidity across the entire theoretical price range. Adams et al. (2021)

To address this issue, Uniswap V3 allows liquidity providers to allocate their capital within a specific *tick range*. In other words, the tick range

¹The numerical examples in this section are adapted from Mohan (2022).

defines the price interval in which the liquidity is active and earns trading fees from swaps executed in the corresponding pool. If the market price moves outside of this range, the liquidity becomes inactive and no longer earns trading fees. Once the price moves back into the range, the liquidity becomes active again and starts earning fees. By concentrating liquidity near the current market price, liquidity providers can increase the capital efficiency of their positions and potentially earn higher fees per unit of capital. This mechanism creates incentives for liquidity providers to allocate their capital to the price ranges where most trading activity occurs. Adams et al. (2021)

In addition, Uniswap V3 introduced multiple fee tiers for the same token pair. Instead of a single fixed fee of 0.30% as in Uniswap V2, pools can be created with different fee levels, including 0.05%, 0.30%, and 1.00%. Adams et al. (2020) This allows liquidity providers to choose fee structures that better reflect the volatility and risk characteristics of the underlying assets Adams et al. (2021).

2.2 Maximal Extractable Value

2.2.1 Definition of MEV

In AMMs such as Uniswap, the execution price of a trade depends on the state of the liquidity pool at the moment of execution. Since each transaction changes the pool reserves, the order in which transactions are processed directly affects their execution prices. This creates opportunities for market participants to profit from the ordering of transactions within a block.

This phenomenon gives rise to *Maximal Extractable Value* (MEV). MEV refers to the value that can be extracted from block production through the inclusion, exclusion, or reordering of transactions within a block Darian et al. (2019); Heimbach et al. (2023).

2.2.2 Types of MEV

MEV can be categorized into three main types. First, *arbitrage* transactions exploit price differences between different DEXs or liquidity pools. By buying an asset on one market and simultaneously selling it on another, arbitrageurs can profit from temporary price discrepancies Qin et al. (2022).

Second, *liquidations* occur on decentralized lending protocols such as Aave. When a borrower’s position becomes undercollateralized, liquidators can repay part of the debt and receive a liquidation bonus as compensation Qin et al. (2022).

Third, *sandwich attacks* exploit the price impact generated by pending transactions on AMM-based DEXs. In a typical sandwich attack, a MEV bot places a frontrun transaction before a victim’s trade and a backrun transaction immediately afterwards. By exploiting the price impact created by the victim’s transaction, the attacker earns a profit between the frontrun and backrun transactions Xu et al. (2023); Zhou et al. (2023). Sandwich attacks are the main focus of this study.

2.2.3 Proposer-Builder Separation (PBS)

Ethereum transitioned from a Proof-of-Work (PoW) to a Proof-of-Stake (PoS) consensus mechanism in September 2022. In the current Ethereum ecosystem, block production is increasingly organized through Proposer-Builder Separation (PBS), which separates the process of identifying MEV opportunities from the process of proposing blocks Heimbach et al. (2023).

The first participants in this process are *searchers*. Searchers continuously monitor the mempool also known as MEV bots, identify profitable MEV opportunities such as arbitrage transactions, liquidations, or sandwich attacks. Once a profitable opportunity is detected, the searcher creates a bundle of transactions and submits it to one or multiple block builders Heimbach et al. (2023). A key property of such bundles is

atomicity: multiple transactions execute in an all-or-nothing sequence, such that if any single transaction fails, all previously executed transactions are reverted without producing a blockchain state change Qin et al. (2022). In practice, searchers often submit multiple competing bundles targeting the same MEV opportunity. Since only one bundle can successfully capture the opportunity, the remaining submissions result in *reverted transactions*. These transactions are included in a block but fail during execution. Although no blockchain state change occurs, the sender is still charged a gas fee for the computational resources consumed up to the point of failure Zhu et al. (2025).

Block builders receive transaction bundles from many searchers and compete to construct the most profitable block possible. Their goal is to maximize the value of a block by selecting and ordering transactions that generate the highest revenue Heimbach et al. (2023).

The constructed block is then submitted to a *relay*. Relays act as intermediaries between builders and validators. They forward block bids while keeping the block contents hidden until a bid has been accepted, preventing validators from copying profitable transaction strategies and building the block themselves Heimbach et al. (2023).

Finally, *validators* receive competing block bids from relays and select the highest bid. The selected block is then proposed to the Ethereum network and becomes part of the blockchain Heimbach et al. (2023).

PBS has led to a specialized ecosystem in which searchers, builders, relays, and validators perform distinct roles in the extraction and distribution of MEV.

2.3 Sandwich Attacks

Sandwich attacks are a form of MEV that commonly occur on DEXs such as Uniswap. They exploit the price impact of a victim’s transaction to generate profit Xu et al. (2023).

Consider a victim transaction T_V that intends to buy WETH using USDC. Since the transaction increases the demand for WETH within the liquidity pool, it is expected to increase the price of WETH. An attacker can exploit this anticipated price movement by placing two transactions around the victim’s trade Zhou et al. (2023).

First, the attacker submits a *frontrun* transaction, denoted by T_{A1} , which is included immediately before the victim’s transaction T_V . By purchasing WETH before the victim, especially when the victim transaction is expected to have a large price impact, the attacker increases the price of WETH in the pool. As discussed in Section 2.1.2, this additional price impact causes the victim’s transaction T_V to be executed at a higher price Daian et al. (2019); Zhou et al. (2023).

After the victim’s transaction, the attacker submits a *backrun* transaction, denoted by T_{A2} , which is included immediately after the victim’s trade T_V . In the backrun transaction, the attacker sells the previously acquired WETH at a higher price and thereby realizes a profit Zhou et al. (2023).

As a result, the attacker profits from the difference between the purchase price in the frontrun transaction T_{A1} and the selling price in the backrun transaction T_{A2} Zhou et al. (2023). This profit represents the extractable value generated through the insertion and reordering of transactions within a block.

While the example above illustrates a buy-side sandwich attack, the same logic applies to sell-side attacks. In a sell-side sandwich attack, the victim intends to sell WETH for USDC, thereby decreasing the price of WETH in the pool. The attacker exploits this by first selling WETH before T_{A1} the victim’s transaction T_V . After the victim’s trade has reduced the pool price, the attacker repurchases WETH through a backrun transaction T_{A2} at a lower price. In both cases, the attacker profits from the price impact caused by the victim’s trade, while the victim receives a worse execution price due to the attacker’s frontrun.

3 Data and Methodology

3.1 Data

The data were collected directly from the Ethereum blockchain via public RPC providers, specifically LlamaRPC and the Ethereum public node. The dataset covers all Uniswap V3 swap events for the WETH/USDC trading pair with a pool fee tier of 0.05%, recorded during the first quarter of 2026 (1 January–31 March 2026). In total, the dataset includes 581,386 swap events distributed across 325,825 blocks. The WETH/USDC 0.05% pool was selected because it is one of the most actively traded pools on Uniswap V3.

Data collection and the empirical analysis were implemented using Python and R scripts. The scripts are available on GitHub.²

Each observation corresponds to a single swap event and contains variables at three levels: the block level, transaction level, and swap level.

At the *block level*, each swap is linked to the corresponding block number, timestamp, base fee, fee recipient, and the block’s extra Data field, which often contains identifiers or labels associated with block builders. The field was used to identify builders in the analysis.

At the *transaction level*, the data include the transaction hash, transaction index, gas used, and effective gas price. Gas costs are calculated by multiplying gas used by the effective gas price, which is reported in Gwei. The resulting gas cost is converted to ETH and then to USD using the realized execution price by the corresponding swap.

At the *swap level*, the data are obtained from the Swap event emitted by the Uniswap V3 pool contract. The relevant information includes the signed token amounts `amount0` (USDC) and `amount1` (WETH). These variables are used to derive trade direction, execution prices, and the USD value of each swap.

²<https://github.com/RafaBala28/mev-sandwich-analysis-uniswap-v3>

In addition, the pool price is derived from the `sqrtPriceX96` variable emitted in each Swap event. Following the Uniswap V3 specification, `sqrtPriceX96` represents the square root of the pool price and reflects the post-trade pool state Adams et al. (2021). This post-trade price is used in the victim damage calculation.

3.2 Sandwich Detection

3.2.1 Detection Process

The detection of sandwich attacks follows a sequential filtering procedure to identify the characteristic transaction pattern of a sandwich attack within a block.

First, blocks containing fewer than three swap events are excluded, since a complete sandwich attack requires at least three swaps: a frontrun transaction, a victim transaction, and a backrun transaction.

Second, attacker candidates are identified as addresses that execute both a buy swap and a sell swap (or vice versa) within the same block. To qualify as a candidate, the two swaps must belong to different transaction hashes. These swaps are treated as potential frontrun and backrun transactions, as sandwich attackers typically open and close a position within the same block.

Third, known router addresses are excluded from the set of attacker candidates, as routers execute trades on behalf of users and are not considered as attackers. In total, five router addresses are filtered out, including three Uniswap routers and two 1inch routers.

Fourth, a valid sandwich attack requires at least one victim swap executed by an address different from the attacker and placed between the frontrun and backrun transactions. The victim swap must occur between the frontrun and backrun transactions within a block. In addition, the victim swap must have the same trade direction as the frontrun transaction.

3.2.2 Sandwich Examples

To illustrate the detection process described in Section 3.2.1 and the resulting attack classification, two detected buy sandwich attacks from the sample are examined in detail. A sell example is provided in the Appendix, see Table 12.

The example shown in Table 1 illustrates a typical buy-side sandwich attack. The TX column reports the transaction position within the block, derived from the transaction’s *tx_index*. The price shown corresponds to the realized execution price of each swap, calculated as the ratio of the exchanged USDC and WETH amounts, and therefore already reflects the Uniswap pool fee of 0.05% and any slippage incurred during execution.

Detected buy-side sandwich attack in Block 24’396’840							
Role	TX	Address	Direction	USDC	WETH	Price	Gas
Frontrun	0	0x6402...b859	BUY	363’595	-192.00	1’893	0.20
Victim	1	0xe592...1564	BUY	400’000	-211.00	1’897	0.58
Backrun	2	0x6402...b859	SELL	-363’921	192.00	1’895	297.00
Other	3	0xe592...a167	SELL	-150’443	79.50	1’893	0.05
Other	4	0xfbd4...4c37	SELL	-231’326	122.00	1’891	0.08

Table 1: All swaps observed in Block 24’396’840 for a detected buy-side sandwich attack with a full close.

The attacker purchases 192 WETH immediately before the victim’s buy order of 400’000 USDC and sells the same amount directly afterwards in the backrun transaction. As a result, the attacker fully closes the position within the same block.

As the table illustrates, the victim pays a higher execution price for WETH than the attacker in the frontrun transaction. The attacker subsequently sells the acquired WETH at a higher price in the backrun transaction, generating extractable value from the price impact caused by the victim’s trade.

The high gas fee paid for the backrun transaction suggests a strong incentive to secure the desired transaction within the block.

The estimated gross profit equals the difference between the USDC re-

ceived in the backrun transaction and the USDC spent in the frontrun transaction. After subtracting gas costs, the estimated net profit of this sandwich attack amounts to approximately \$29.

Table 2 shows a different pattern. As in the previous example, a frontrun and a backrun transaction can be observed. The difference is that the backrun is split into two separate swap events sharing the same transaction hash, indicating that both swaps were executed within the same transaction.

Detected buy-side sandwich attack in Block 24'139'045							
Role	TX	Address	Direction	USDC	WETH	Price	Gas
Frontrun	0	0x1f2f...f387	BUY	169'211	-56.60	2'989	0.03
Victim	1	0x68b3...fc45	BUY	50'000	-16.70	2'996	0.07
Backrun	3	0x1f2f...f387	SELL	-119'681	40.10	2'988	74.20
Backrun	3	0x1f2f...f387	SELL	-15'800	5.30	2'983	74.10
Other	5	0xa7da...2155	SELL	-6'134	2.06	2'982	0.10
Other	44	0x313e...8ebc	SELL	-303	0.102	2'981	0.02
Other	55	0xa77f...f740	SELL	-228	0.076	2'981	0.03

Table 2: All swaps observed in Block 24'139'045 for a detected buy-side sandwich attack with a partial close.

The two backrun swap events display slightly different gas costs in USD. This difference arises because gas costs are converted from ETH to USD using the respective execution price of each swap event. Importantly, both swap events belong to the same Ethereum transaction and therefore share a single gas payment.

A second important observation concerns the amount of WETH sold in the backrun relative to the amount purchased in the frontrun transaction. The attacker buys 56.6 WETH but sells only 45.4 WETH across both backrun swap events, leaving approximately 11.2 WETH unclosed after the block has been finalized.

A third observation is that both backrun swap events are executed at a worse price than the frontrun transaction. This shows that sandwich attacks are not necessarily profitable when considering only the observed swap prices within the block.

3.2.3 Backrun Classification

Table 2 illustrates that sandwich attacks are not always fully closed within the same block. For this reason, sandwich attacks are classified into different categories based on the relationship between the amount of WETH bought in the frontrun transaction and the amount of WETH sold in the backrun transaction.

The classification is based on the following ratio:

$$r = \frac{|\text{WETH}_b|}{|\text{WETH}_f|} \quad (3)$$

where WETH_f denotes the amount of WETH bought in the frontrun transaction, and WETH_b denotes the amount of WETH sold in the backrun transaction.

Full close refers to sandwich attacks in which the attacker sells approximately the same amount of WETH in the backrun transaction as was bought in the frontrun transaction, implying $r \approx 1$. In this study, a sandwich attack is classified as a full close if $0.99 \leq r \leq 1.01$.

Partial close describes sandwich attacks in which the attacker closes only part of the frontrun position in the backrun transaction, implying $r < 0.99$.

Over close occurs when the attacker sells more WETH in the backrun transaction than was bought in the frontrun transaction, implying $r > 1.01$.

3.3 Profit Calculation

The sandwich attack classification directly affects the profit calculation. As shown by the partial close example in Section 3.2.2, frontrun and backrun transactions cannot always be compared one-to-one. The underlying idea is that only the portion of WETH realized within the sandwich attack is included in the profit measure. Any inventory that remains

unclosed after the backrun transaction, as well as inventory that was already held by the attacker prior to the attack in over close cases, is treated as unrealized and excluded from the calculation.

If a backrun transaction contains multiple swap events, these are aggregated at the transaction-hash level. The WETH and USDC amounts used in the profit calculation therefore correspond to the sum of all swap events associated with the backrun transaction. Each sandwich attack is associated with exactly one backrun transaction hash. Only this transaction is included in the profit calculation. Any additional swaps executed by the same attacker within the same block but associated with different transaction hashes are excluded.

To account for partial and over close cases, the close ratio r is used to adjust the profit calculation according to the realized fraction of the position.

The profit calculation is based on the realized swap amounts observed on-chain. Consequently, the calculated gross profit already reflects the effects of the Uniswap pool fee of 0.05% and any execution slippage.

Gross profit is defined as:

$$\pi^g = \begin{cases} |\text{USDC}_b| - \text{USDC}_f, & \text{if } r \approx 1 \quad (\text{Full Close}), \\ |\text{USDC}_b| - r \cdot \text{USDC}_f, & \text{if } r < 0.99 \quad (\text{Partial Close}), \\ \frac{|\text{USDC}_b|}{r} - \text{USDC}_f, & \text{if } r > 1.01 \quad (\text{Over Close}). \end{cases} \quad (4)$$

For partial closes ($r < 0.99$), only the realized fraction of the frontrun position is considered. For over closes ($r > 1.01$), the backrun proceeds are scaled to the size of the original frontrun position. This approach ensures that profitability is measured only for the inventory directly associated with the sandwich attack.

Net profit is obtained by subtracting the transaction costs of the frontrun

and backrun transactions from gross profit:

$$\pi^n = \pi^g - (\text{Gas}_f + \text{Gas}_b) \quad (5)$$

where Gas_f denotes the gas costs of the frontrun transaction and Gas_b denotes the gas costs of the backrun transaction, both expressed in USD. The conversion from ETH to USD is based on the execution price observed in the corresponding swap.

As discussed previously, a backrun transaction may contain multiple swap events sharing the same transaction hash. Since gas costs are incurred at the transaction level, the attacker pays transaction fees only once for the entire backrun transaction. Therefore, all swap amounts are aggregated at the transaction-hash level, while the gas cost of the backrun transaction is represented by the average USD gas cost across the associated swap events. This avoids double-counting gas.

3.4 Victim Loss Measures

In this study, *victim damage* is defined as the additional trading cost incurred by a victim as a result of a sandwich attack. The underlying assumption is that, without the attacker’s frontrun transaction, the victim would have been able to execute the swap at the pool price that existed immediately before the frontrun, which corresponds to the pool price resulting from the last swap before the attack. Victim damage is therefore measured as the difference between the pool price before the frontrun transaction and the pool price after the frontrun transaction, multiplied by the amount of WETH traded by the victim:

$$\text{Damage}_v = |p_v - p_f| \cdot |\text{WETH}_v| \quad (6)$$

where p_v denotes the Uniswap pool price immediately after the attacker’s frontrun transaction and therefore the pool price faced by the victim, and p_f denotes the pool price immediately before the attacker’s frontrun transaction. The resulting measure represents the estimated additional

cost to the victim due to the sandwich attack.

A second measure considered in this study is *victim slippage*, defined as the additional trading cost resulting from the victim’s own price impact within the AMM. It is measured as the difference between the victim’s execution price and the Uniswap pool price immediately before the victim transaction was executed, multiplied by the amount of WETH traded by the victim:

$$\text{Slippage}_v = |p_e - p_v| \cdot |\text{WETH}_v| \quad (7)$$

where p_e denotes the victim’s realized execution price, p_v denotes the Uniswap pool price immediately before the victim’s transaction and therefore the pool price faced by the victim, and WETH_v denotes the amount of WETH traded by the victim.

The *total loss* is defined as the total execution loss experienced by the victim relative to the pool price that existed before the attacker’s frontrun transaction. It combines the effects of sandwich damage and victim slippage:

$$\text{Total Loss}_v = |p_e - p_f| \cdot |\text{WETH}_v| \quad (8)$$

where p_e denotes the victim’s realized execution price, p_f denotes the Uniswap pool price immediately before the attacker’s frontrun transaction, and WETH_v denotes the amount of WETH traded by the victim.

3.5 Limitations

Several limitations apply to the analysis conducted in this study.

First, the profit calculation is based on the swap amounts observed within the identified sandwich attack. Any WETH inventory held by the attacker prior to the frontrun transaction or carried forward after the backrun transaction is not observable and is therefore excluded from the profit calculation. Similarly, infrastructure costs such as development and maintenance expenses associated with running a MEV bot are not captured.

Second, reverted transactions are not included in the analysis. This may lead to an underestimation of total attack attempts and associated costs.

Third, bot profitability is measured at the address level. Potential connections between different attacker addresses, such as addresses controlled by the same entity, are outside the scope of this study.

Fourth, each sandwich attack is required to have exactly one transaction hash associated with the backrun transaction. Cases with more complex backrun structures are excluded.

Fifth, the analysis is restricted to the Ethereum mainnet and a single liquidity pool. Sandwich attacks occurring across multiple pools or on other blockchain networks are not considered.

Finally, the sandwich detection algorithm is based on observed transaction patterns. As a result, some complex transaction sequences may not be classified correctly, and false positives or false negatives may occur.

4 Empirical Results

4.1 Overview and Descriptive Statistics

In total, 699 sandwich attacks were detected in the WETH/USDC 0.05% Uniswap V3 pool during Q1 2026.

Figure 1 presents the daily number of detected attacks over the same period, together with the corresponding daily ETH price.

While the average number of detected sandwich attacks was 7.8 per day, activity was strongly concentrated in early February. The highest number of attacks was observed on February 6, 2026, with 84 detected sandwich attacks, followed by February 5, 2026, with 80 attacks. These peaks occurred immediately after a sharp decline in the ETH price, which fell from approximately USD 3'000 to around USD 2'000 within a few days.

Daily Frequency of Detected Sandwich Attacks and ETH Price



Figure 1: Daily number of detected sandwich attacks in the Uniswap V3 WETH/USDC 0.05% pool during Q1 2026. The grey bars represent the number of detected sandwich attacks per day, while the black line shows the daily ETH price in USD. The dashed horizontal line marks the average daily attack frequency.

At the same time, these days also show high trading volume. On February 5, the pool processed approximately USD 1.15 billion and on February 6, the volume reached approximately USD 830 million. Over the entire observation period, the pool recorded a total trading volume of approximately USD 16.23 billion, corresponding to an average daily volume of about USD 180.30 million.

The Spearman correlation indicates a moderate positive association between daily trading volume and the number of detected sandwich attacks ($\rho = 0.400$, $p < 0.001$, $n = 90$), suggesting that higher trading volume is related to more sandwich attacks, although trading volume alone does not fully explain sandwich activity.

4.2 Classification of Sandwich Attacks

Table 3 reports the distribution of sandwich attacks across directions and backrun types. For the 699 detected sandwich attacks in the dataset, 363

were classified as buy-side attacks and 336 as sell-side attacks. Only 21 attacks (3.0%) were classified as full closes, of which 8 occurred on the buy side and 13 on the sell side. A detailed list of all identified full-close attacks is provided in Appendix Table 11. Partial closes represent the largest category, accounting for 464 attacks (66.4%), while over closes account for the remaining 214 attacks (30.6%).

Sandwich Attacks by Side and Backrun Type				
Side	Total Attacks	Type	Attacks	Share
Buy-side	363	Full close	8	1.1%
		Partial close	237	33.9%
		Over close	118	16.9%
Sell-side	336	Full close	13	1.9%
		Partial close	227	32.5%
		Over close	96	13.7%
Total	699			100.0%

Table 3: Sandwich attacks grouped by direction and backrun type. Share denotes the fraction of total detected sandwich attacks ($N = 699$).

The small share of full closes suggests that most sandwich attacks are not fully closed within the same block and within the observed Uniswap V3 pool. In the majority of cases, the attacker still holds a remaining position after the backrun transaction.

On the other hand, over close attacks indicate that bots sell more in the backrun than they purchased in the frontrun. The available data do not reveal where the additional inventory in over close attacks comes from, whether from previous partial closes or other MEV strategies. This suggests that sandwich attacks are often part of a larger strategy. Further on-chain investigation would be required.

4.3 Bot Profitability

4.3.1 Attack Size and Profitability by Backrun Type

Table 4 reports attack size and profitability across backrun types to examine whether full, partial, and over close attacks differ.

Sandwich Attacks by Backrun Type						
Type	Attacks	Med. FR	Med. Gas	Med. Net Profit	Tot. Gas	Tot. Profit
Full close	21	161'427	20.50	16.60	3'541	3'201
Partial close	464	49'216	4.71	-9.15	58'650	-50'919
Over close	214	1'090	3.34	-4.65	6'669	-5'006
Total	699	14'347	4.17	-6.45	68'860	-52'724

Table 4: Distribution of detected sandwich attacks by backrun type. Med. denotes median values, FR denotes the frontrun size, and Tot. denotes total values. Monetary values are reported in USD.

The results show clear differences across backrun types. Full close attacks are considerably larger, with a median frontrun size of USD 161'427, compared to USD 49'216 for partial closes and USD 1'090 for over closes. Gas costs also differ substantially across backrun types. Full close attacks have a median gas cost of USD 20.50, compared to USD 4.71 for partial closes and USD 3.34 for over closes.

In total, full close attacks generate a net profit of USD 3'201 while incurring gas costs of USD 3'541, making them the only profitable category overall. In contrast, partial closes and over closes generate total net losses of USD -50'919 and USD -5'006.

At the individual attack level, 81.0% of full closes are profitable. In contrast, only 6.9% of partial closes and 5.6% of over closes generate positive net profits, indicating that most attacks in these categories fail to cover their gas costs.

4.3.2 Profit Decomposition

Table 4 showed that sandwich attacks were loss-making in aggregate, generating a total net loss of USD -52'724 during Q1 2026. Table 5

provides an analysis of theoretical AMM profits, pool fees, slippage, and gas costs to explain the results presented in Section 4.3.1.

Profit Decomposition by Backrun Type						
Type	AMM Profit	Pool Fees	Slippage	Gross Profit	Gas	Net Profit
Full close	52'603	6'130	39'731	6'742	3'541	3'201
Partial close	819'349	56'004	755'614	7'732	58'650	-50'919
Over close	7'258	2'217	3'378	1'663	6'669	-5'006
Total	879'210	64'351	798'723	16'135	68'860	-52'724

Table 5: Profit decomposition by backrun type. AMM Profit denotes the theoretical profit based on Uniswap pool prices implied by the pool state. Gross Profit equals AMM Profit minus Pool Fees and Slippage. Net Profit equals Gross Profit minus Gas. Monetary values are reported in USD.

Table 5 provides a deeper look at the profit structure. Based on theoretical AMM prices, total AMM profit across all attacks amounts to USD 879'210, implying that sandwich attacks are highly profitable in theory. Slippage eliminates approximately 90.8% of total AMM profit, reducing theoretical profits by USD 798'723. Pool fees charged by the Uniswap V3 pool account for an additional USD 64'351 in costs. After deducting both slippage and pool fees, only USD 16'135 of gross profit remains. Once gas costs of USD 68'860 are subtracted, the overall result is a net loss of USD -52'724.

This shows that while sandwich attacks generate substantial theoretical value, slippage alone eliminates more than 90% of potential profits. The remaining gross profit is insufficient to cover gas costs, resulting in a net loss overall.

4.3.3 Top 10 Attackers

The dataset contains 59 unique attacker addresses, indicating a high degree of concentration. Table 6 shows that the two most active attackers alone account for 388 of the 699 detected attacks (55.5%), while the top 10 attackers combined are responsible for 568 attacks (81.3%). This confirms that sandwich activity is highly concentrated among a small number of participants. This pattern is further supported by a Gini coefficient of 0.79 (see Figure 3 in the Appendix).

Top 10 Sandwich Attackers by Number of Detected Attacks

Address	Attacks	Gross Profit	Gas	Net Profit
0x66a9...a8af	219	-717	1'309	-2'026
0x1f2f...f387	169	5'559	57'767	-52'208
0xf48a...3f1c	47	-196	573	-769
0xaf68...1f7d	38	17	68	-50
0x6324...6cb0	21	-76	396	-473
0xfbd4...4c37	20	2'788	2'612	176
0x6dc7...8a7f	19	515	234	282
0x263d...52e4	12	707	12	695
0x6747...acb5	12	-40	105	-145
0x1f56...71a5	11	-2	182	-184
Total (Top 10)	568	8'555	63'258	-54'703

Table 6: Top 10 attacker addresses ranked by the number of detected sandwich attacks. Gross profit reflects realized trading profit, including swap fees and slippage. Net profit is calculated after subtracting frontrun and backrun gas costs. All monetary values are reported in USD.

Only 3 of the top 10 attackers generate a positive net profit. The two most active attackers, 0x66a9...a8af with 219 attacks and 0x1f2f...f387 with 169 attacks, report net losses of USD -2'026 and USD -52'208, respectively. For 0x1f2f...f387, the large net loss is a consequence of gas costs of USD 57'767.

Looking at all 59 attacker addresses, only 14 (23.7%) are net profitable overall, suggesting that while a small number of MEV bots operate profitably, the majority of sandwich attackers fail to cover their gas and slippage costs.

4.4 Victim Loss

4.4.1 Sandwich Damage

The sample contains 968 victim swaps across 180 unique victim addresses, representing approximately 0.17% of all swaps observed in the pool. Since one attack can target multiple victims, the number of victim swaps ex-

ceeds the number of detected attacks (see Appendix Table 10). Victim swaps are larger than typical swaps, with a median size of USD 5’971 compared to USD 2’088 across all swaps.

As defined in Section 3.4, sandwich damage measures the additional trading cost imposed on victims by the attacker’s frontrun transaction. Across all victim transactions, total estimated sandwich damage amounts to USD 193’641. Table 7 reports the distribution of estimated victim damage per transaction.

Distribution of Sandwich Damage (USD)						
	Min	Q1	Median	Mean	Q3	Max
Damage	0.00	0.00	0.21	200	38.9	19’678

Table 7: Distribution of estimated victim damage per victim transaction.

The distribution of victim damage is highly right-skewed. While the mean damage is USD 200, the median damage is only USD 0.21, indicating that most victims experience relatively small damage. This is further reflected by the first quartile of USD 0.00 and the third quartile of USD 38.90.

As a result, only a small number of transactions suffer substantial losses. The maximum observed damage reaches USD 19’678. These large outliers raise the average damage well above the median.

In relative terms, total sandwich damage amounts to 0.28% of total victim trading volume of USD 68.7 million. While the aggregate impact remains limited, individual victims can still suffer substantial losses, as reflected by the maximum observed damage.

4.4.2 Victim Slippage and Execution Loss

Following Section 3.4, total execution loss is decomposed into two components: sandwich damage, which captures the additional cost imposed by the attacker’s frontrun transaction, and victim slippage, which reflects the cost arising from the victim’s own price impact within the AMM.

Victim Execution Loss Decomposition

Metric	Damage (USD)	Share of Total
Sandwich damage	193'641	59.9%
Victim slippage	129'471	40.1%
Total Loss	323'112	100%

Table 8: Decomposition of total victim execution loss into sandwich damage and victim slippage.

Table 8 shows the total execution losses suffered by victims during the observation period. Total execution losses amount to USD 323'112. Of this amount, USD 193'641 (59.9%) is caused by sandwich attacks, while the remaining USD 129'471 (40.1%) results from the victim's own slippage. This shows that execution losses are not caused only by sandwich attacks. However, sandwich attacks remain the largest source of losses, accounting for 59.9% of total execution losses.

4.4.3 Top 10 Victims

Table 9 shows the ten most affected victim addresses by total sandwich damage. Compared to the attacker side, the distribution of attacks across victims is considerably less concentrated. The most frequently targeted address, `0xe592...1564`, was involved in 142 attacks and suffered total damage of USD 85'250. This address corresponds to the Uniswap V3 Router contract, meaning that the underlying victims are individual users routing their trades through the protocol. Similarly, `0x68b3...fc45` is an aggregator contract.

The addresses in Table 9 represent recorded sender addresses. To reduce router classifications, known router addresses were replaced by the swap recipient when possible. Some router contracts still appeared among the largest victims, suggesting that the recorded recipient is not always the final user. Identifying the actual victims would require further transaction-level analysis beyond the scope of this study.

Top 10 Victim Addresses by Estimated Sandwich Damage

Address	Attacks	Tot. Volume	Tot. Damage	Damage (%)	Med. Damage
0xe592...1564	142	33'717'398	85'250	0.25%	241.00
0x66a9...a8af	92	14'126'183	54'620	0.39%	13.80
0x9906...d086	24	2'316'208	16'216	0.70%	1.84
0x6324...6cb0	26	1'248'857	10'438	0.84%	2.46
0x083c...9998	1	821'076	7'426	0.90%	7'426.00
0x68b3...fc45	92	4'025'947	6'655	0.17%	60.50
0xb441...6fda	7	564'801	4'793	0.85%	847.00
0xd2b3...612b	6	225'275	1'814	0.81%	1.32
0xf48a...3f1c	28	1'291'352	1'360	0.11%	0.15
0x3be1...95b8	6	26'107	1'099	4.21%	0.00

Table 9: Top 10 victim addresses ranked by estimated Total sandwich damage. Damage (%) is calculated as total damage divided by total transaction volume. Monetary values are reported in USD.

A notable observation is that three addresses appearing in the top 10 victim table also appear among the top 10 attackers in Table 6: `0x66a9...a8af`, `0x6324...6cb0`, and `0xf48a...3f1c`. More broadly, 38 of the 59 identified attacker addresses (64.4%) also appear as victim addresses elsewhere in the sample. At the swap level, this concerns 442 of 968 victim swaps. This overlap suggests a competitive MEV environment in which addresses associated with sandwich attacks may themselves become targets.

This result should be interpreted with caution. Some manual checks show a clear sandwich structure, but not all cases are simple bot-versus-bot sandwiches. In some cases, there are multiple backrun swaps, several victims, or additional transactions between the frontrun and backrun.

4.5 Attacker-Builder Relationships

After focusing on victims in the previous section, the analysis now turns to the relationship between attackers and block builders. Figure 2 shows the relationship between the ten most active sandwich attackers and block builders. Titan included 415 of the 699 detected sandwich attacks (59.4%), making it the dominant builder in the sample. To show whether sandwich activity is concentrated on specific builders, builder shares in detected sandwich attacks were compared with builder shares in the full

swap sample (see Appendix Table 13). Titan accounted for 58.0% of all swaps and 59.4% of detected sandwich attacks. This is largely consistent with its overall market share. Bob, by contrast, accounted for only 0.7% of all swaps but 3.0% of detected sandwich attacks, a pattern that may warrant further investigation.

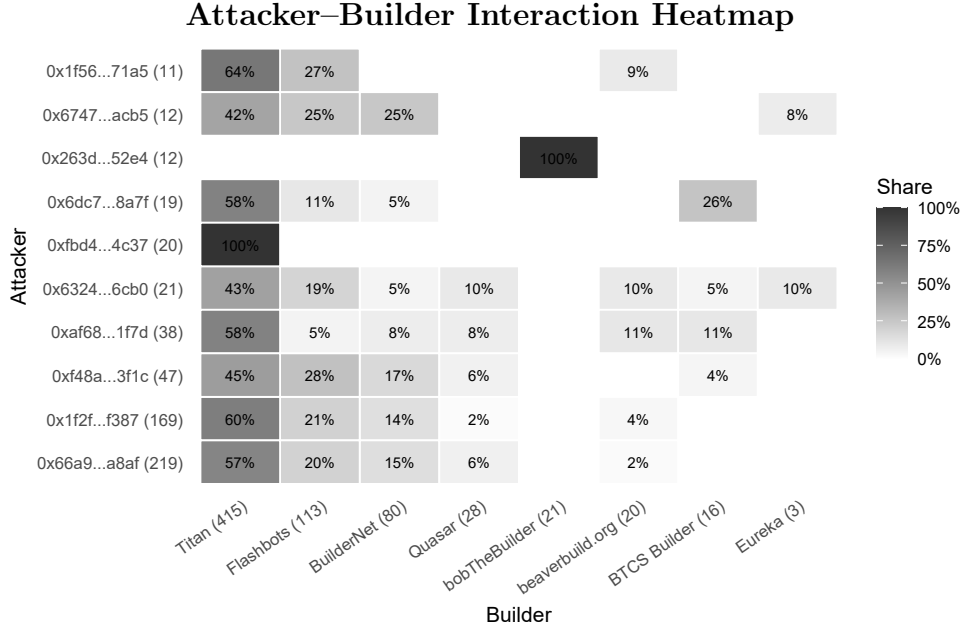


Figure 2: Interaction patterns between the ten most active sandwich attackers and builders. Percentages indicate the share of each attacker’s attacks included by a particular builder. Values in parentheses denote the total number of detected attacks per attacker. Darker shading indicates a higher share of interactions.

A notable observation is that `0x263d...52e4` exclusively uses `bobTheBuilder` (100%). This may indicate a preference for this builder, although the small number of observations limits the strength of this interpretation. Most other attackers distribute their attacks across multiple builders, with Titan, Flashbots, and BuilderNet being the most commonly used builders. Most MEV bots use other builders. This suggests that submitting bundles to multiple builders is a common strategy to increase the chance of inclusion. Titan’s dominant position may indicate that it has competitive advantages over competing builders.

Table 14 in the Appendix reports median gas costs across attacker–builder pairs. Attackers with exclusive builder relationships show very different

cost levels. For example, `0x263d...52e4` exclusively uses `bobTheBuilder` and pays a median gas cost of only USD 0.98, while `0xfbd4...4c37` exclusively uses `Titan` and pays USD 60.20. Furthermore, gas costs vary considerably across attackers using the same builder. For `Titan`, `0x66a9...a8af` pays a median of USD 2.59, while `0x1f2f...f387` pays USD 44.20. This suggests that some attackers are able to secure block inclusion at lower cost than others, possibly reflecting differences in bundle optimization, bidding strategies, or relationships with specific builders. Direct payments or other forms of compensation may also play a role, but these mechanisms are not observable in the data. In addition, a single searcher may control multiple addresses. Address clustering would therefore be needed to identify searcher-level behaviour more accurately.

5 Conclusion

5.1 Summary of Findings

RQ1: The findings indicate that sandwich attacks occur regularly in the Uniswap V3 WETH/USDC pool. A total of 699 attacks were detected during the first quarter of 2026. These attacks involved 968 victim swaps, corresponding to approximately 0.17% of all swaps observed in the pool. Attack activity increased during periods of elevated market volatility and trading volume, suggesting that sandwich attacks become more prevalent when market conditions create greater opportunities for MEV extraction.

The analysis further shows that only 21 of the 699 detected attacks (3.0%) were fully closed within the same block. Most attacks were classified as partial closes (66.4%) or over closes (30.6%), suggesting that sandwich attacks are often more complex than the typical full close examples commonly discussed in the literature and may represent only one component of larger strategies.

RQ2: The profitability analysis shows that sandwich attacks were not profitable in aggregate during the observation period. Although the at-

tacks generated theoretical profits of USD 879'210 based on Uniswap pool prices, these gains largely disappeared once actual execution costs were taken into account. As a result, sandwich attackers collectively realized a net loss of USD 52'724, and only 14 of the 59 identified attacker addresses were profitable overall.

Full close attacks were the only category that was profitable in aggregate, whereas partial close and over close attacks generated net losses overall. The analysis showed that theoretical MEV opportunities do not necessarily translate into realized profits. Slippage eliminated most theoretical AMM profits before gas costs were taken into account.

RQ3: The results show that victim losses must be differentiated between losses caused by sandwich attacks and losses arising from the AMM mechanism itself. Total execution losses amounted to USD 323'112, of which USD 193'641 (59.9%) were attributable to sandwich attacks and USD 129'471 (40.1%) resulted from the victims' own slippage. This shows that execution losses are not caused only by MEV but also by the AMM pricing mechanism through slippage. Still, sandwich attacks remained the largest source of execution losses in the sample.

RQ4: The analysis reveals a high degree of concentration among both sandwich attackers and block builders. Only 59 unique attacker addresses were identified in the dataset, with the two most active attackers alone accounting for 55.5% of all detected attacks. The concentration becomes even more pronounced when considering the ten most active attackers, which together were responsible for 81.3% of all sandwich attacks. This pattern is further supported by a Gini coefficient of 0.79, indicating that sandwich activity is highly concentrated among a small number of participants. Since the analysis is conducted at the address level, address clustering could reveal an even higher concentration if several addresses are controlled by the same searcher.

A similarly concentrated structure can be observed on the builder side. Titan was the dominant builder, including 415 of the 699 detected sandwich attacks. The results further suggest that some searchers main-

tain persistent relationships with specific builders, while others distribute their bundles across multiple builders. In addition, substantial differences in gas costs can be observed across attackers using the same builder. This indicates that certain attackers are able to secure block inclusion at significantly lower cost, potentially reflecting differences in bidding strategies, bundle optimization, or direct relationships with builders.

5.2 Implications

The findings of this study have implications for several stakeholders within the DeFi ecosystem.

For traders, the results highlight that sandwich attacks are a persistent source of execution costs. At the same time, slippage is not only relevant for victims, but also for attackers. Both sides can face substantial slippage costs due to the AMM pricing mechanism, especially for large trades. This highlights the importance of trade size, slippage tolerance, and execution conditions when using AMM-based DEXs.

Users should therefore consider MEV protection tools, such as Flashbots Protect or MEV Blocker. It sends transactions to a private Flashbots mempool, where they are hidden from frontrunning and sandwich bots. Flashbots (2025); MEV Blocker (2025)

A particularly interesting result is that sandwich attacks create clear losers but no obvious winners. Victims incur substantial losses, but attackers do not earn these losses as profits because slippage, pool fees, and gas costs absorb the value. This suggests that a share of the losses is transferred to other participants in the ecosystem, such as liquidity providers through swap fees and block producers through gas fees.

In the PBS system, builders are not passive actors, as they decide which bundles and transactions are included in a potential next block. In principle, builders could reject bundles containing clear sandwich patterns, since these strategies impose predictable losses on users and raise ethical concerns. However, the empirical results show that all top 10 builders

in the sample included sandwich attacks in their blocks. This points to a potential conflict in the current PBS architecture, where builders face economic incentives to maximize block value rather than to protect users from harmful MEV strategies.

5.3 Future Research

Future research can be divided into three main areas. First, the results show that sandwich attackers actively manage inventory rather than fully closing their trades within the same block. The high share of partial close and over close attacks suggests that sandwich attacks may represent only one component of broader trading strategies. Future research could investigate how MEV bots manage inventory across blocks and pools and whether sandwich attacks are linked to other forms of MEV extraction, such as arbitrage or liquidations.

Second, the analysis highlights the important role of slippage for both victims and attackers. While victim slippage accounted for a substantial share of total execution losses, slippage was also the main factor reducing the profitability of sandwich attacks. Future research could investigate how different trading strategies, pool characteristics, or protocol designs affect slippage and whether its impact can be reduced. In particular, future research could examine how traders can better protect themselves against high slippage.

Third, the results reveal substantial differences in gas costs across attackers and builders. Some attackers appear to secure block inclusion at significantly lower costs than others, while certain searchers exhibit strong preferences for specific builders. Future research could investigate whether these differences are driven solely by bidding and bundle optimization strategies or whether closer relationships between searchers and builders exist. A better understanding of these mechanisms would provide further insights into competition within the PBS ecosystem.

References

- Adams, H., Zinsmeister, N. and Robinson, D. (2020), ‘Uniswap V2’.
URL: <https://app.uniswap.org/whitepaper.pdf>
- Adams, H., Zinsmeister, N., Robinson, D. and Konstantopoulos, G. (2021), ‘Uniswap v3 Core’.
URL: <https://app.uniswap.org/whitepaper-v3.pdf>
- Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L. and Juels, A. (2019), ‘Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges’. arXiv:1904.05234 [cs].
URL: <http://arxiv.org/abs/1904.05234>
- DefiLlama (2026a), ‘DefiLlama - DeFi Dashboard & Crypto Analytics’.
URL: <https://defillama.com>
- DefiLlama (2026b), ‘Uniswap TVL, Fees, Revenue & Volume’.
URL: <https://defillama.com/protocol/uniswap>
- Flashbots (2025), ‘MEV Protection Overview | Flashbots Docs’.
URL: <https://docs.flashbots.net/flashbots-protect/overview>
- Heimbach, L., Kiffer, L., Ferreira Torres, C. and Wattenhofer, R. (2023), Ethereum’s Proposer-Builder Separation: Promises and Realities, in ‘Proceedings of the 2023 ACM on Internet Measurement Conference’, IMC ’23, Association for Computing Machinery, New York, NY, USA, pp. 406–420.
URL: <https://dl.acm.org/doi/10.1145/3618257.3624824>
- MEV Blocker (2025), ‘Introduction | MEV Blocker documentation’.
URL: <https://docs.mevblocker.io/>
- Mohan, V. (2022), ‘Automated market makers and decentralized exchanges: a DeFi primer’, *Financial Innovation* **8**(1), 20.
URL: <https://jfin-swufe.springeropen.com/articles/10.1186/s40854-021-00314-5>

Qin, K., Zhou, L. and Gervais, A. (2022), Quantifying Blockchain Extractable Value: How dark is the forest?, *in* ‘2022 IEEE Symposium on Security and Privacy (SP)’, pp. 198–214.

URL: <https://ieeexplore.ieee.org/document/9833734/>

Wang, Y., Zuest, P., Yao, Y., Lu, Z. and Wattenhofer, R. (2022), Impact and User Perception of Sandwich Attacks in the DeFi Ecosystem, *in* ‘Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems’, CHI ’22, Association for Computing Machinery, New York, NY, USA, pp. 1–15.

URL: <https://dl.acm.org/doi/10.1145/3491102.3517585>

Xu, J., Paruch, K., Cousaert, S. and Feng, Y. (2023), ‘SoK: Decentralized Exchanges (DEX) with Automated Market Maker (AMM) Protocols’, *ACM Computing Surveys* **55**(11), 1–50. arXiv:2103.12732 [q-fin].

URL: <http://arxiv.org/abs/2103.12732>

Zhou, L., Xiong, X., Ernstberger, J., Chaliasos, S., Wang, Z., Wang, Y., Qin, K., Wattenhofer, R., Song, D. and Gervais, A. (2023), SoK: Decentralized Finance (DeFi) Attacks, *in* ‘2023 IEEE Symposium on Security and Privacy (SP)’, pp. 2444–2461. ISSN: 2375-1207.

URL: <https://ieeexplore.ieee.org/document/10179435/>

Zhu, B. Z., Wan, X., Moallemi, C. C., Robinson, D. and Bachu, B. (2025), ‘Quantifying the Value of Revert Protection’. arXiv:2410.19106 [cs.GT].

URL: <http://arxiv.org/abs/2410.19106>

Declaration of AI Assistance

ChatGPT and Claude Code were used as coding, language, and editing assistants to improve wording, structure, and grammar. The empirical analysis, data collection, code implementation, interpretation of results, and final responsibility for the content remain with the author.

Plagiatserklärung

Ich bezeuge mit meiner Unterschrift, dass ich meine Arbeit selbständig ohne fremde Hilfe verfasst habe und meine Angaben über die bei der Abfassung meiner Arbeit benützten Quellen in jeder Hinsicht der Wahrheit entsprechen und vollständig sind. Alle Quellen, die wörtlich oder sinngemäss übernommen wurden, habe ich als solche gekennzeichnet.

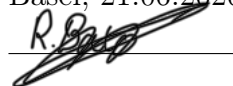
Des Weiteren versichere ich, sämtliche Textpassagen, die unter Zuhilfenahme KI-gestützter Programme verfasst wurden, entsprechend gekennzeichnet sowie mit einem Hinweis auf das verwendete KI-gestützte Programm versehen zu haben.

Eine Überprüfung der Arbeit auf Plagiate und KI-gestützte Programme – unter Einsatz entsprechender Software – darf vorgenommen werden. Ich habe zur Kenntnis genommen, dass unlauteres Verhalten zu einer Bewertung der betroffenen Arbeit mit einer Note 1 oder mit «nicht bestanden» bzw. «fail» oder zum Ausschluss vom Studium führen kann.

Ich habe das Merkblatt zu Plagiat und Betrug vom 3. März 2022 gelesen und bin mir der Konsequenzen eines solchen Handelns bewusst.

Rafael Balasteguim da Silva

Basel, 21.06.2026



6 Appendix

Number of Victim Swaps per Sandwich Attack

Victim Swaps	Attacks	Share
1	568	81.3%
2	81	11.6%
3	25	3.6%
4	9	1.3%
5	4	0.6%
6+	12	1.7%

Table 10: Distribution of victim swaps per detected sandwich attack.

Full-Close Sandwich Attacks

Block	Side	Attacker	Victim	Victim Volume	Gross Profit	Gas	Net Profit
24'355'997	SELL-side	0x01fd...f8e1	0xb441...6fda	77'410	2'330	4	2'326
24'392'437	SELL-side	0x6dc7...8a7f	0x66a9...a8af	913'270	613	2	611
24'289'601	SELL-side	0x01fd...f8e1	0xe592...1564	83'834	456	342	114
24'543'377	BUY-side	0x027b...1114	0xe592...1564	619'548	586	529	57
24'367'814	SELL-side	0x01fd...f8e1	0x9906...d086	19'561	36	1	35
24'466'057	SELL-side	0x1f2f...f387	0x9906...d086	438'628	635	603	32
24'396'840	BUY-side	0x6402...b859	0xe592...1564	400'000	326	297	29
24'453'736	BUY-side	0x1f2f...f387	0xe592...1564	754'063	492	468	24
24'545'259	SELL-side	0x1f2f...f387	0x66a9...a8af	447'724	643	618	24
24'395'237	BUY-side	0x6402...b859	0xe592...1564	200'000	216	197	19
24'340'093	BUY-side	0x1f2f...f387	0x68b3...fc45	50'018	17	0	17
24'356'942	SELL-side	0x01fd...f8e1	0x66a9...a8af	24'628	25	15	11
24'396'828	BUY-side	0x6402...b859	0xe592...1564	300'000	146	143	3
24'139'131	BUY-side	0x1f2f...f387	0x68b3...fc45	50'014	20	20	0
24'395'320	BUY-side	0x6402...b859	0xe592...1564	200'000	21	20	0
24'379'605	SELL-side	0x1f2f...f387	0xe592...1564	45'000	234	234	0
24'380'698	SELL-side	0xc30f...2252	0x3650...b5b8	19'999	0	0	0
24'254'072	SELL-side	0xc30f...2252	0x66a9...a8af	8'331	-4	0	-4
24'440'621	SELL-side	0x66a9...a8af	0x51c7...2a7f	23'614	-1	4	-5
24'365'732	SELL-side	0x1f2f...f387	0x66a9...a8af	15'379	15	44	-29
24'586'745	SELL-side	0x0906...9e63	0x6324...6cb0	44'032	-63	0	-63

Table 11: Identified full-close sandwich attacks. Full-close attacks are defined as observations with a close ratio between 0.99 and 1.01. Monetary values are reported in USD.

Detected SELL-side sandwich attack in Block 24'348'957

Role	TX	Address	Direction	USDC	WETH	Price	Gas
Frontrun	0	0x1f2f...f387	SELL	-1'951'016	726.00	2'686	4.19
Victim	1	0xe592...1564	SELL	-88'190	34.00	2'595	1.43
Backrun	2	0x1f2f...f387	BUY	1'863'431	-694.00	2'687	2'609.00
Backrun	2	0x1f2f...f387	BUY	18'762	-6.87	2'732	2'653.00
Other	7	0xfbd4...4c37	BUY	22'877	-8.37	2'733	2.40
Other	27	0x5050...2edf	BUY	4'354	-1.59	2'733	0.38
Other	30	0x51c7...2a7f	BUY	15'504	-5.67	2'734	0.98

Table 12: All swaps observed in Block 24'348'957 for a detected sell-side partial-close sandwich attack.

Lorenz Curve of Sandwich Attack Concentration

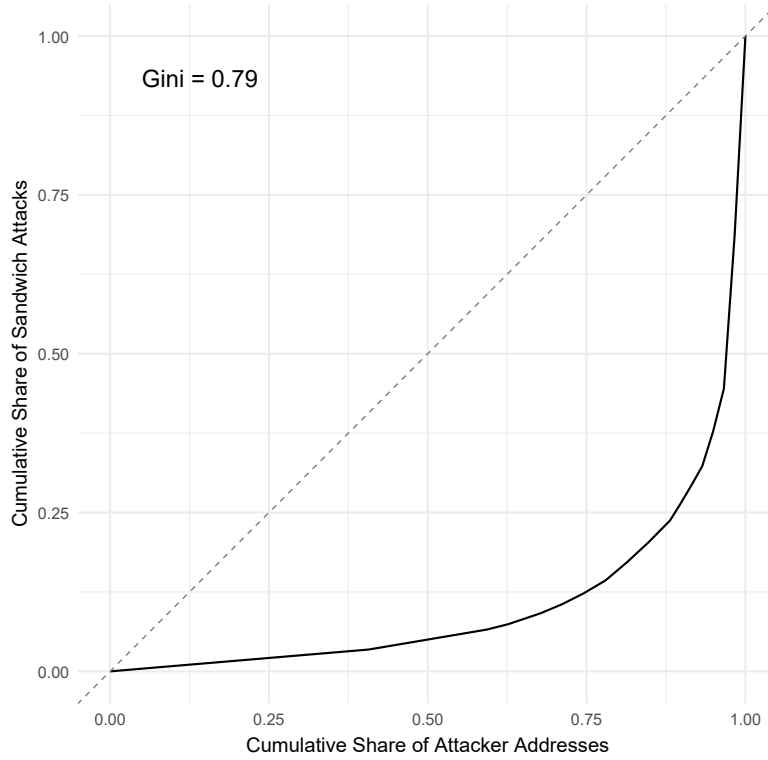


Figure 3: Lorenz curve showing the distribution of detected sandwich attacks across attacker addresses. The dashed 45-degree line represents perfect equality, while the solid line depicts the observed distribution of sandwich activity. The Gini coefficient of 0.79 quantifies the degree of concentration among attackers.

Builder Shares in All Swaps and Detected Sandwich Attacks

Builder	Swaps	Share (%)	Sandwiches	Share (%)
Titan	336'916	58.0	415	59.4
Flashbots	88'473	15.2	113	16.2
BuilderNet	72'331	12.4	80	11.4
Quasar	52'477	9.0	28	4.0
Bob	4'008	0.7	21	3.0
Beaver	11'407	2.0	20	2.9
BTCS	5'752	1.0	16	2.3
Unknown	5'568	1.0	3	0.4
Eureka	3'384	0.6	3	0.4
Nethermind	1'070	0.2	0	0.0

Table 13: Distribution of all swaps and detected sandwich attacks across builders. Shares are reported relative to the total number of swaps and detected sandwich attacks, respectively.

Median Gas

Costs (USD) Across Builders for the Ten Most Active Attackers

Attacker	Total	Beaver	BuilderNet	Flashbots	Quasar	Titan	Bob	BTCS	Eureka
0x66a9...a8af	219	2.50	2.36	1.88	1.89	2.59	—	—	—
0x1f2f...f387	169	128.00	26.50	34.60	31.80	44.20	—	—	—
0xf48a...3f1c	47	—	3.21	2.85	11.30	4.37	—	3.75	—
0xaf68...1f7d	38	1.04	0.10	0.15	0.14	0.29	—	0.33	—
0x6324...6cb0	21	0.50	0.27	7.26	1.79	8.00	—	10.30	0.65
0xfbd4...4c37	20	—	—	—	—	60.20	—	—	—
0x6dc7...8a7f	19	—	13.90	1.23	—	0.70	—	1.63	—
0x263d...52e4	12	—	—	—	—	—	0.98	—	—
0x6747...acb5	12	—	11.70	6.34	—	8.09	—	—	1.13
0x1f56...71a5	11	60.10	—	10.20	—	8.56	—	—	—

Table 14: Median combined gas costs of frontrun and backrun transactions for the ten most active attackers. Columns report median gas costs across builders. Empty cells indicate that no attacks were observed for the corresponding attacker-builder pair.